



DOCUMENTAÇÃO VENDEPAY

Webhooks

Guia de integração

Informação clara e confiável para aproveitar os recursos da plataforma VendePay.

Neste guia

1. Configure sua integração	2
Antes de começar	2
Obtenha a URL de recebimento	2
Cadastre o webhook na VendePay	2
Salve o secret	3
2. Teste e valide a conexão	4
O que confirmar no destino	4
3. Escolha os eventos	4
4. Use os dados recebidos	5
Campos usados com frequência	5
Campos ausentes ou vazios	6
5. Referência de payloads	6
Como ler os schemas	6
Pagamentos e checkout	6
Carrinho abandonado	6
Pix gerado	7
Boleto gerado	8
Compra recusada	9
Compra aprovada	10
Reembolso	11
Chargeback	12
Assinaturas	13
Assinatura iniciada	13
Assinatura renovada	14
Grace period iniciado	15
Assinatura cancelada	16
6. Entrega e segurança	17
Formato da requisição	17
Valide o secret	18
Resposta esperada	18
Comportamento da entrega	18
7. Solução de problemas	18
8. Valores de referência	19
Método de pagamento	19
Moeda da venda	19
Status da venda	19
Status do checkout	20

Este guia mostra como conectar os webhooks da VendePay à sua plataforma de automação, sistema ou aplicação. Comece pela configuração e pelo teste; consulte os payloads completos apenas quando precisar mapear campos ou resolver dúvidas.

1. Configure sua integração

Antes de começar

Prepare:

- Uma URL pública de recebimento, fornecida pela sua plataforma de automação, sistema ou aplicação.
- A lista de eventos e produtos que deseja acompanhar.
- Um local seguro para guardar o secret apresentado pela VendePay.

O fluxo completo é simples:

Obtenha a URL de recebimento

- Cadastre o webhook na VendePay
- Salve o secret
- Envie um teste
- Use os dados recebidos na sua automação

Obtenha a URL de recebimento

Na plataforma ou sistema que receberá os eventos, crie um novo receptor de webhook e copie a URL pública fornecida. Essa é a URL que você cadastrará na VendePay.

O receptor deve aceitar requisições POST com dados no formato JSON. Mantenha essa URL privada, pois ela permite acionar a integração.

Cadastre o webhook na VendePay

No menu lateral do painel VendePay, selecione **Extensões**.

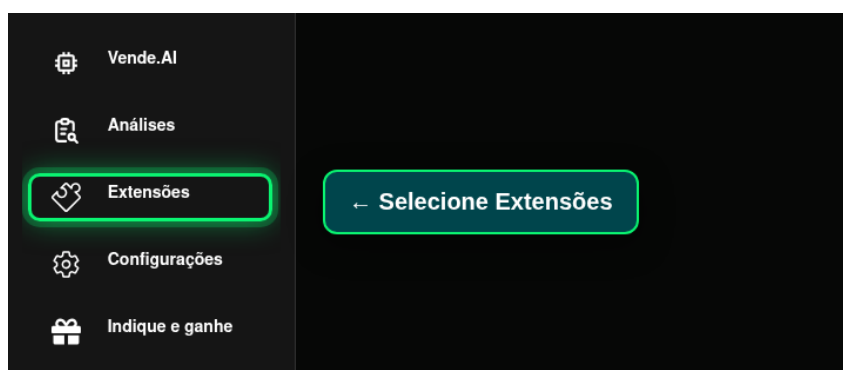


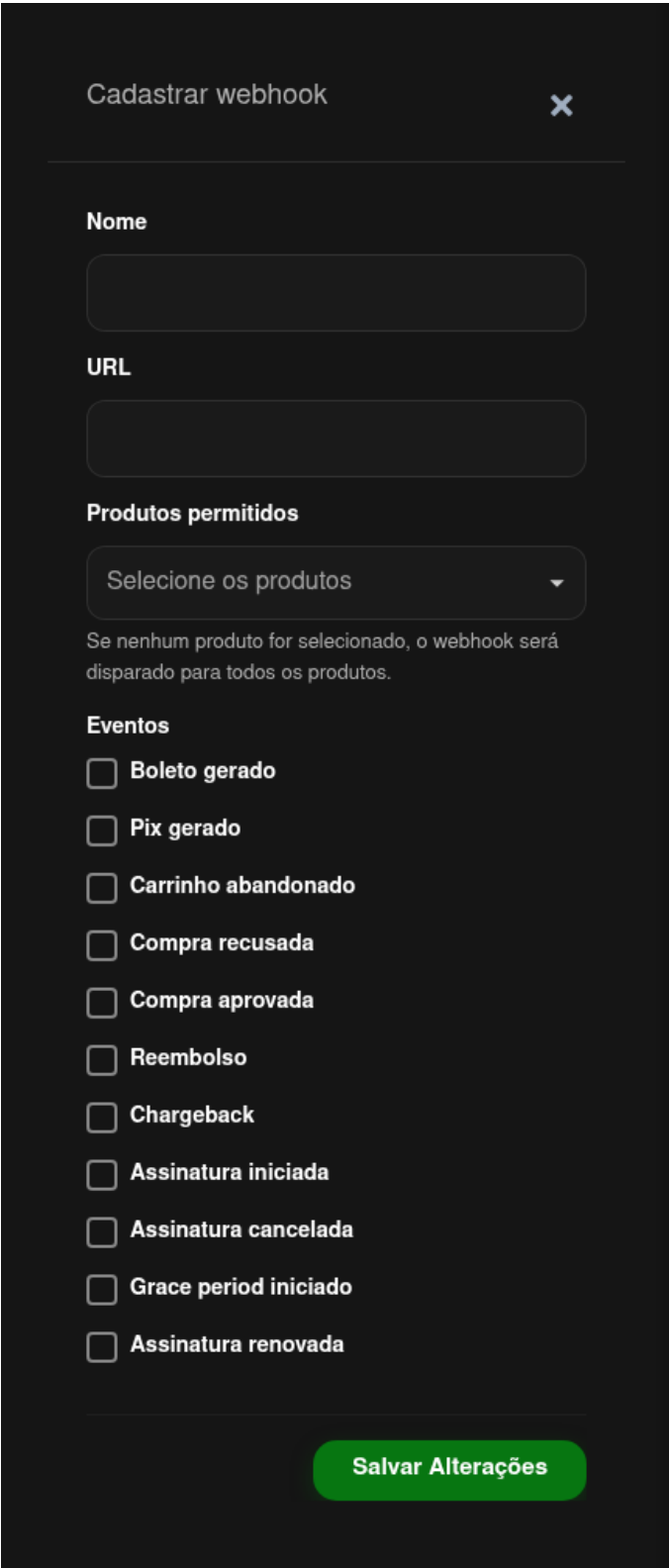
Figura 1 — Acesso à área de extensões da plataforma.

Na seção **Integrações**, localize **Webhooks**, selecione **Configurar** e, na tela seguinte, clique em **Nova integração**.

O que configurar

- **Nome:** identificação interna para reconhecer o webhook.
- **URL:** endereço público copiado da plataforma ou sistema de destino.
- **Produtos permitidos:** produtos cujos eventos devem ser enviados. Sem uma seleção, o webhook será usado para todos os produtos.
- **Eventos:** notificações que deseja receber nesse endpoint.

Depois de revisar as opções, selecione **Salvar Alterações**.



O formulário, intitulado "Cadastrar webhook", possui um botão de fechar no canto superior direito. Ele contém os seguintes campos e opções:

- Nome:** Campo de texto para a identificação interna.
- URL:** Campo de texto para o endereço público.
- Produtos permitidos:** Menu suspenso com o texto "Selecione os produtos". Abaixo dele, uma explicação: "Se nenhum produto for selecionado, o webhook será disparado para todos os produtos."
- Eventos:** Lista de eventos com checkboxes para seleção:
 - ☐ Boleto gerado
 - ☐ Pix gerado
 - ☐ Carrinho abandonado
 - ☐ Compra recusada
 - ☐ Compra aprovada
 - ☐ Reembolso
 - ☐ Chargeback
 - ☐ Assinatura iniciada
 - ☐ Assinatura cancelada
 - ☐ Grace period iniciado
 - ☐ Assinatura renovada

No rodapé do formulário, há um botão verde com o texto "Salvar Alterações".

Figura 2 — Configuração da URL, dos produtos e dos eventos do webhook.

Salve o secret

Após o cadastro, a VendePay apresenta o secret do webhook. Selecione **Copiar** e guarde o valor em um local seguro antes de fechar a tela.

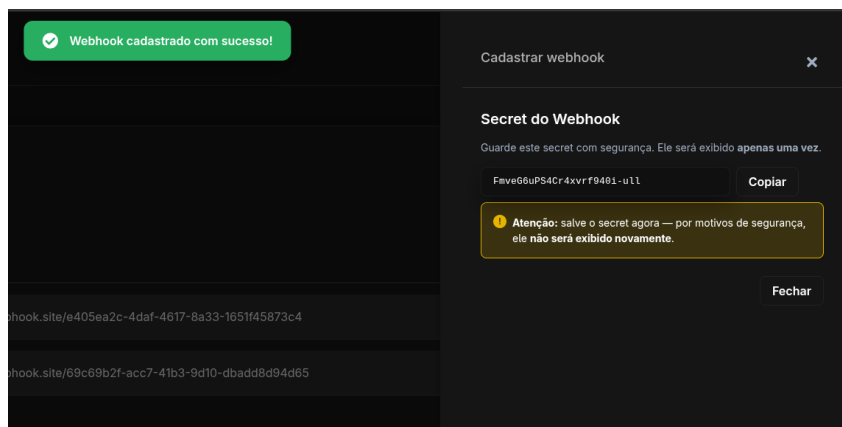


Figura 3 — Secret apresentado após o cadastro do webhook.

O secret permite verificar se uma chamada foi enviada pela VendePay. Não o exponha em páginas públicas, mensagens ou logs.

2. Teste e valide a conexão

Na lista de webhooks, abra as ações da integração e selecione **Enviar teste**. Escolha um dos eventos configurados e confirme o envio.

O que confirmar no destino

Na plataforma ou sistema que recebe o webhook, verifique se:

- Uma nova requisição foi recebida.
- O método recebido é POST.
- O body foi reconhecido como JSON.
- O campo `event` identifica o evento selecionado.
- O header `x-signature-key` contém o secret salvo durante o cadastro.
- A integração respondeu com um status HTTP de sucesso (2xx).

Requisições geradas por **Enviar teste** também incluem:

```
X-Webhook-Test: true
```

Se esses itens estiverem corretos, a conexão básica está funcionando e você já pode mapear os campos recebidos para as próximas etapas da sua automação.

Sobre os dados de teste: os valores são sintéticos. Use o teste para configurar a integração e reconhecer os campos, mas consulte a referência deste guia antes de assumir que um campo sempre estará preenchido em uma entrega real.

3. Escolha os eventos

Selecione apenas os eventos necessários para o seu processo. Isso simplifica a automação e evita execuções desnecessárias.

Os nomes abaixo correspondem aos exibidos na plataforma. O identificador entregue aparece no campo `event` do body e no header `X-Webhook-Event`.

Pagamentos e checkout

- **Boleto gerado** (`boleto.gerado`): uma venda por boleto é criada. Use para acompanhar pagamentos pendentes ou iniciar comunicações.
- **Pix gerado** (`pix.gerado`): uma venda por PIX é criada. Use para acompanhar pagamentos pendentes ou iniciar comunicações.
- **Carrinho abandonado** (`carrinho.abandonado`): o checkout permanece sem pagamento após o período de abandono. Use em fluxos de recuperação.
- **Compra recusada** (`compra.recusada`): uma tentativa de cartão é recusada em um fluxo suportado. Use em comunicações ou análises de falha.
- **Compra aprovada** (`compra.aprovada`): a compra é aprovada e seus recebíveis são gerados. Use para liberar acesso, atualizar CRM ou iniciar o pós-venda.
- **Reembolso** (`reembolso`): um reembolso passa pelo processamento financeiro. Use para atualizar sistemas e revisar acessos ou benefícios.
- **Chargeback** (`chargeback`): um chargeback é processado. Use para atualizar sistemas, acessos e rotinas de atendimento.

Assinaturas

- **Assinatura iniciada** (`assinatura.realizada`): uma assinatura é iniciada. Use para liberar ou registrar o acesso recorrente.
- **Assinatura cancelada** (`assinatura.cancelada`): uma assinatura é cancelada. Use para atualizar acesso, CRM e comunicações.
- **Grace period iniciado** (`grace.period.iniciado`): uma assinatura entra no período de tolerância. Use em comunicações e recuperação de pagamento.
- **Assinatura renovada** (`Assinatura renovada`): uma renovação é paga. Use para estender acesso e atualizar a vigência da assinatura.

Os identificadores diferenciam letras maiúsculas e minúsculas. Em especial, `Assinatura renovada` começa com letra maiúscula e contém um espaço.

4. Use os dados recebidos

Cada entrega contém duas partes principais:

- **Headers:** informações sobre a entrega, como o tipo do evento e o secret do webhook.
- **Body:** dados do evento em formato JSON, como compra, cliente, produto, checkout ou assinatura.

Em uma automação, o fluxo normalmente segue esta sequência:

```
Webhook recebido
→ Identifique o evento
→ Filtre pelo produto ou status
→ Leia os dados necessários
→ Execute a ação no sistema de destino
```

Campos usados com frequência

Os nomes variam entre os payloads. Consulte o evento específico antes de criar o mapeamento definitivo.

- **Tipo do evento:** `event`, para escolher o caminho da automação.
- **Compra ou tentativa:** `id` e `vendaId`, para relacionar e atualizar registros.
- **Produto:** `produtoId` e `checkout.produtoId`, para filtrar produtos e liberar o acesso correto.
- **Cliente:** `emailComprador`, `comprador.email` e `nomeComprador`, para localizar ou criar um contato.
- **Valor e moeda:** `valorPago`, `valor` e `moeda`, para registrar informações financeiras.
- **Checkout:** `checkoutId` e `checkout.id`, para relacionar a jornada de compra.
- **Assinatura:** `assinaturaId` e `assinatura.id`, para atualizar vigência e acesso recorrente.

Campos ausentes ou vazios

Nem todo campo estará preenchido em todas as entregas. Ao configurar filtros e ações:

- Trate campos opcionais como possivelmente ausentes.
- Trate campos anuláveis como possivelmente iguais a `null`.
- Aceite novos campos sem interromper a automação.
- Não dependa da ordem das propriedades no JSON.

5. Referência de payloads

Você não precisa ler toda esta seção para configurar o webhook. Consulte o evento necessário quando for mapear campos, confirmar formatos ou investigar uma entrega.

Como ler os schemas

Os schemas usam uma notação semelhante a TypeScript:

```
campo: string;           // esperado no payload
campo: string | null;    // pode ser null
campo?: string;          // pode ser omitido
```

UUIDs e datas são representados como texto; datas seguem o formato ISO 8601. A integração também deve aceitar propriedades adicionais que não estejam apresentadas nos schemas.

`Record<string, string>` representa um objeto com chaves e valores textuais variáveis. `Record<string, unknown>` identifica um objeto complementar cuja estrutura pode variar e não deve ser usada como contrato fixo.

Pagamentos e checkout

Os eventos desta seção acompanham a jornada do checkout, a criação de instruções de pagamento e os estados posteriores de uma compra.

Carrinho abandonado

Quando é enviado: quando o tempo de abandono termina e o checkout ainda não foi pago.

Identificador: `carrinho.abandonado`

Dados mais usados: `checkoutId`, `comprador.email`, `checkout.produtoId`, `checkout.total` e `ultimoEvento`.

Payload de referência:

```
type CarrinhoAbandonadoPayload = {
  event: "carrinho.abandonado";
  trackeamentoId: string;
  checkoutId: string;
  ip: string | null;
  urlParams: Record<string, string>;
  ultimoEvento: string | null;
  createdAt: string;

  comprador: {
    nome: string | null;
    sobrenome: string | null;
    email: string | null;
    telefone: string | null;
    doc1: string | null;
  }
}
```

```
    doc2: string | null;
  };

  endereco: {
    rua: string | null;
    bairro: string | null;
    cidade: string | null;
    estado: string | null;
    pais: string | null;
    cep: string | null;
  };

  checkout: {
    id: string;
    status: number;
    moeda: number;
    pais: string | null;
    valor: number | null;
    subtotal: number | null;
    total: number | null;
    produtoId: string;
    afiliadoId: string | null;
    linguagem: number | string | null;
    cupomId: string | null;
    aplicouCupom: boolean;
  };

  nomeProduto?: string;
  descricaoProduto?: string;
};
```

Pix gerado

Quando é enviado: após a criação de uma venda por PIX.

Identificador: pix.gerado

Dados mais usados: id, produtoId, emailComprador, valorPago, status e checkoutId.

Payload de referência:

```
type PixGeradoPayload = {
  event: "pix.gerado";
  id: string;
  createdAt: string;
  produtoId: string;
  cupomDescontoId: string | null;
  vendedorId: string;
  checkoutId: string;
  idepotentialCheckoutId: string;

  emailComprador: string;
  nomeComprador: string;
  sobrenomeComprador: string;
  telefoneComprador: string;
  cpfComprador: string;

  metodoPagamento: 1;
  moeda: number;
  status: number;
  handledByGlobalChargeback: boolean;
  valorPago: number;
  postalCode: string | null;
  state: string | null;
  city: string | null;
```

```
district: string | null;
address: string | null;
number: string | null;
complement: string | null;

approveUrl: string | null;
clientSecret: string | null;
nextReconcileAt: string | null;
lastReconcileAt: string | null;
reconcileAttempts: number;
reconcileStatus: number;
recebiveisOk: boolean;

src: string | null;
sck: string | null;
utmSource: string | null;
utmCampaign: string | null;
utmMedium: string | null;
utmContent: string | null;
utmTerm: string | null;
urlParams: Record<string, string> | null;

nomeProduto?: string | null;
descricaoProduto?: string | null;
};
```

Boleto gerado

Quando é enviado: após a criação de uma venda por boleto.

Identificador: boleto.gerado

Dados mais usados: id, produtoId, emailComprador, valorPago, status e checkoutId.

Payload de referência:

```
type BoletoGeradoPayload = {
  event: "boleto.gerado";
  id: string;
  createdAt: string;
  produtoId: string;
  cupomDescontoId: string | null;
  vendedorId: string;
  checkoutId: string;
  idepotentialCheckoutId: string;

  emailComprador: string;
  nomeComprador: string;
  sobrenomeComprador: string;
  telefoneComprador: string;
  cpfComprador: string;

  metodoPagamento: 2;
  moeda: number;
  status: number;
  handledByGlobalChargeback: boolean;
  valorPago: number;
  postalCode: string | null;
  state: string | null;
  city: string | null;
  district: string | null;
  address: string | null;
  number: string | null;
  complement: string | null;
};
```

```

approveUrl: string | null;
clientSecret: string | null;
nextReconcileAt: string | null;
lastReconcileAt: string | null;
reconcileAttempts: number;
reconcileStatus: number;
recebiveisOk: boolean;

src: string | null;
sck: string | null;
utmSource: string | null;
utmCampaign: string | null;
utmMedium: string | null;
utmContent: string | null;
utmTerm: string | null;
urlParams: Record<string, string> | null;

nomeProduto?: string | null;
descricaoProduto?: string | null;
};

```

Compra recusada

Quando é enviado: quando uma tentativa de cartão é recusada durante o processamento suportado.

Identificador: compra.recusada

Dados mais usados: id, produtoId, checkoutId, emailComprador, valorPago e metodoPagamento.

Payload de referência:

```

type CompraRecusadaPayload = {
  event: "compra.recusada";
  id: string;

  produtoId: string;
  vendedorId: string;
  checkoutId: string;
  metodoPagamento: number;
  moeda: number;
  valorPago: number;

  emailComprador: string;
  nomeComprador: string;
  sobrenomeComprador: string;
  telefoneComprador?: string;
  installments: number;

  nomeProduto: string;
  descricaoProduto: string;

  afiliadoId?: string;
  ofertaId?: string;
  cupomDesconto?: string;

  postalCode?: string;
  state?: string;
  city?: string;
  district?: string;
  address?: string;
  number?: string;
  complement?: string;

  paymentMethodId?: string;
  issuer?: number;
};

```

```
src?: string;
sck?: string;
utmSource?: string;
utmCampaign?: string;
utmMedium?: string;
utmContent?: string;
utmTerm?: string;
urlParams?: Record<string, string>;
};
```

Observação de integração: o campo id identifica a tentativa recusada e pode não corresponder a uma venda persistida.

Compra aprovada

Quando é enviado: após a aprovação e geração dos recebíveis da venda.

Identificador: compra.aprovada

Dados mais usados: id, produtoId, emailComprador, valorPago, status e metodoPagamento.

Payload de referência:

```
type CompraAprovadaPayload = {
  event: "compra.aprovada";
  id: string;
  createdAt: string;
  produtoId: string;
  cupomDescontoId: string | null;
  vendedorId: string;
  checkoutId: string;
  idepotentialCheckoutId: string;

  emailComprador: string;
  nomeComprador: string;
  sobrenomeComprador: string;
  telefoneComprador: string;

  cpfComprador?: string;

  metodoPagamento: number;
  moeda: number;
  status: number;
  handledByGlobalChargeback: boolean;
  valorPago: number;
  postalCode: string | null;
  state: string | null;
  city: string | null;
  district: string | null;
  address: string | null;
  number: string | null;
  complement: string | null;

  approveUrl: string | null;
  clientSecret: string | null;
  nextReconcileAt: string | null;
  lastReconcileAt: string | null;
  reconcileAttempts: number;
  reconcileStatus: number;
  recebeveisOk: boolean;

  src: string | null;
  sck: string | null;
  utmSource: string | null;
  utmCampaign: string | null;
```

```

utmMedium: string | null;
utmContent: string | null;
utmTerm: string | null;
urlParams: Record<string, string> | null;

nomeProduto?: string | null;
descricaoProduto?: string | null;
};

```

Variação possível: o reenvio manual disponível nos detalhes da venda pode omitir `cpfComprador`. A integração deve tratar esse campo como opcional.

Reembolso

Quando é enviado: quando o reembolso passa pelo processamento financeiro correspondente.

Identificador: reembolso

Dados mais usados: `id`, `produtoId`, `emailComprador`, `valorPago`, `status` e `metodoPagamento`.

Payload de referência:

```

type ReembolsoPayload = {
  event: "reembolso";
  id: string;
  createdAt: string;
  produtoId: string;
  cupomDescontoId: string | null;
  vendedorId: string;
  checkoutId: string;
  idepotentialCheckoutId: string;

  emailComprador: string;
  nomeComprador: string;
  sobrenomeComprador: string;
  telefoneComprador: string;
  cpfComprador: string;

  metodoPagamento: number;
  moeda: number;
  status: number;
  handledByGlobalChargeback: boolean;
  valorPago: number;
  upSell: boolean;

  postalCode: string | null;
  state: string | null;
  city: string | null;
  district: string | null;
  address: string | null;
  number: string | null;
  complement: string | null;

  approveUrl: string | null;
  clientSecret: string | null;
  nextReconcileAt: string | null;
  lastReconcileAt: string | null;
  reconcileAttempts: number;
  reconcileStatus: number;
  recebeIeIsOk: boolean;

  src: string | null;
  sck: string | null;
  utmSource: string | null;
  utmCampaign: string | null;
  utmMedium: string | null;

```

```

utmContent: string | null;
utmTerm: string | null;
urlParams: Record<string, string> | null;

nomeProduto?: string | null;
descricaoProduto?: string | null;

vendedor: Record<string, unknown>;
};

```

Objeto complementar: vendedor acompanha a entrega atual, mas seus campos internos não fazem parte do contrato público. Não use esse objeto como origem de dados da integração.

Chargeback

Quando é enviado: quando um chargeback é processado.

Identificador: chargeback

Dados mais usados: id, produtoId, emailComprador, valorPago, status e metodoPagamento.

Payload de referência:

```

type ChargebackPayload = {
  event: "chargeback";
  id: string;
  createdAt: string;
  produtoId: string;
  cupomDescontoId: string | null;
  vendedorId: string;
  checkoutId: string;
  idepotentialCheckoutId: string;

  emailComprador: string;
  nomeComprador: string;
  sobrenomeComprador: string;
  telefoneComprador: string;
  cpfComprador: string;

  metodoPagamento: number;
  moeda: number;
  status: number;
  handledByGlobalChargeback: boolean;
  valorPago: number;
  upSell: boolean;

  postalCode: string | null;
  state: string | null;
  city: string | null;
  district: string | null;
  address: string | null;
  number: string | null;
  complement: string | null;

  approveUrl: string | null;
  clientSecret: string | null;
  nextReconcileAt: string | null;
  lastReconcileAt: string | null;
  reconcileAttempts: number;
  reconcileStatus: number;
  recebeIeIsOk: boolean;

  src: string | null;
  sck: string | null;
  utmSource: string | null;
  utmCampaign: string | null;
}

```

```

utmMedium: string | null;
utmContent: string | null;
utmTerm: string | null;
urlParams: Record<string, string> | null;

nomeProduto?: string | null;
descricaoProduto?: string | null;

vendedor: Record<string, unknown>;
};

```

Objeto complementar: vendedor acompanha a entrega atual, mas seus campos internos não fazem parte do contrato público. Não use esse objeto como origem de dados da integração.

Assinaturas

Os eventos desta seção acompanham o início, a renovação, o período de tolerância e o cancelamento de uma assinatura.

Assinatura iniciada

Quando é enviado: quando uma assinatura é iniciada.

Identificador: assinatura.realizada

Dados mais usados: id, produtoId, emailComprador, valorPago e status.

Payload de referência:

```

type AssinaturaIniciadaPayload = {
  event: "assinatura.realizada";
  id: string;
  createdAt: string;
  produtoId: string;
  cupomDescontoId: string | null;
  vendedorId: string;
  checkoutId: string;
  idepotentialCheckoutId: string;

  emailComprador: string;
  nomeComprador: string;
  sobrenomeComprador: string;
  telefoneComprador: string;
  cpfComprador: string;

  metodoPagamento: number;
  moeda: number;
  status: number;
  handledByGlobalChargeback: boolean;
  valorPago: number;
  postalCode: string | null;
  state: string | null;
  city: string | null;
  district: string | null;
  address: string | null;
  number: string | null;
  complement: string | null;

  approveUrl: string | null;
  clientSecret: string | null;
  nextReconcileAt: string | null;
  lastReconcileAt: string | null;
  reconcileAttempts: number;
  reconcileStatus: number;
  recebeIsOk: boolean;
};

```

```

src: string | null;
sck: string | null;
utmSource: string | null;
utmCampaign: string | null;
utmMedium: string | null;
utmContent: string | null;
utmTerm: string | null;
urlParams: Record<string, string> | null;

nomeProduto?: string | null;
descricaoProduto?: string | null;
};

```

Observação de integração: o payload representa a venda inicial associada à assinatura.

Assinatura renovada

Quando é enviado: quando uma invoice de renovação é paga.

Identificador: Assinatura renovada

Dados mais usados: id, assinaturaId, vendaId, valor e moeda. O recebimento deste evento indica que a renovação foi paga.

Payload de referência:

```

type AssinaturaRenovadaPayload = {
  event: "Assinatura renovada";

  id: string;
  assinaturaId: string;
  vendaId: string | null;
  valor: number;
  moeda: string;
  status: "pending" | "past_due" | "failed" | "processing";
  dueAt: string;
  paidAt: string | null;

  authorizationCode: string | null;
  last4: string | null;
  providerTransactionId: string | null;
  acquirerReferenceNumber: string | null;
  providerTransactionAt: string | null;
  merchantDescriptor: string | null;
  providerPayloadSanitized: Record<string, unknown> | null;
  paymentDetailsCapturedAt: string | null;
  providerAmount: number | null;
  providerCurrency: string | null;

  tentativas: number;
  createdAt: string;
  updatedAt: string;
  lastAttemptAt: string | null;
  nextAttemptAt: string | null;
  gracePeriodEndsAt: string | null;
  recebeIsOk: boolean;

  nomeProduto: string;
  descricaoProduto: string;

  assinatura: {
    id: string;
    planoId: string;
    vendaInicialId: string;
    status: "trialing" | "active" | "past_due" | "canceled";
  };
};

```

```

currentPeriodStartAt: string;
currentPeriodEndAt: string;
cancelAtPeriodEnd: boolean;
canceledAt: string | null;
motivoCancelamento: string | null;
gatewaySubscriptionId: string | null;
metadata: unknown | null;
createdAt: string;
updatedAt: string;

plano: {
  id: string;
  productId: string;
  nome: string;
  descricao: string | null;
  valor: number;
  moeda: number;
  ativo: boolean;
  createdAt: string;
  updatedAt: string;

  billingConfig: Record<string, unknown> | null;
  retryConfig: Record<string, unknown> | null;
  graceConfig: Record<string, unknown> | null;
};
};

venda: Record<string, unknown>;
};

```

Observações de integração:

- O evento é baseado na invoice de renovação e pode incluir dados relacionados à assinatura, ao plano e à venda.
- No payload atual, status, paidAt, updatedAt, os detalhes de pagamento e recebíveis0k representam o snapshot carregado antes da confirmação. Use o próprio evento como confirmação da renovação, não esses campos de estado.
- moeda é texto, mas normalmente contém o código numérico da moeda, como "1" para BRL. Registros anteriores também podem usar códigos como "BRL".
- Os objetos relacionados podem conter propriedades adicionais.
- Dependendo preferencialmente dos campos de invoice apresentados na raiz e trate as relações como dados complementares.

Grace period iniciado

Quando é enviado: quando uma assinatura entra no período de tolerância.

Identificador: grace.period.iniciado

Dados mais usados: id, produtoId, emailComprador, valorPago e moeda.

Payload de referência:

```

type GracePeriodIniciadoPayload = {
  event: "grace.period.iniciado";
  id: string;
  createdAt: string;
  produtoId: string;
  cupomDescontoId: string | null;
  vendedorId: string;
  checkoutId: string;
  idepotentialCheckoutId: string;

  emailComprador: string;
  nomeComprador: string;
  sobrenomeComprador: string;
}

```

```

telefoneComprador: string;
cpfComprador: string;

metodoPagamento: number;
moeda: string;
status: number;
handledByGlobalChargeback: boolean;
valorPago: number;
upSell: boolean;

postalCode: string | null;
state: string | null;
city: string | null;
district: string | null;
address: string | null;
number: string | null;
complement: string | null;

approveUrl: string | null;
clientSecret: string | null;
nextReconcileAt: string | null;
lastReconcileAt: string | null;
reconcileAttempts: number;
reconcileStatus: number;
recebiveisOk: boolean;

src: string | null;
sck: string | null;
utmSource: string | null;
utmCampaign: string | null;
utmMedium: string | null;
utmContent: string | null;
utmTerm: string | null;
urlParams: Record<string, string> | null;

nomeProduto?: string | null;
descricaoProduto?: string | null;
};

```

Variação possível: o evento parte da venda inicial, mas usa a moeda e o valor da invoice que originou o período de tolerância. A moeda é enviada como texto, normalmente com o código numérico, por exemplo, "1" para BRL. Registros anteriores também podem usar "BRL".

Assinatura cancelada

Quando é enviado: quando uma assinatura é cancelada.

Identificador: assinatura.cancelada

Dados mais usados: id, produtoId, emailComprador, valorPago e moeda.

Payload de referência:

```

type AssinaturaCanceladaPayload = {
  event: "assinatura.cancelada";
  id: string;
  createdAt: string;
  produtoId: string;
  cupomDescontoId: string | null;
  vendedorId: string;
  checkoutId: string;
  idepotentialCheckoutId: string;

  emailComprador: string;
  nomeComprador: string;
};

```

```

sobrenomeComprador: string;
telefoneComprador: string;
cpfComprador: string;

metodoPagamento: number;
moeda: number | string;
status: number;
handledByGlobalChargeback: boolean;
valorPago: number;
upSell: boolean;

postalCode: string | null;
state: string | null;
city: string | null;
district: string | null;
address: string | null;
number: string | null;
complement: string | null;

approveUrl: string | null;
clientSecret: string | null;
nextReconcileAt: string | null;
lastReconcileAt: string | null;
reconcileAttempts: number;
reconcileStatus: number;
recebiveisOk: boolean;

src: string | null;
sck: string | null;
utmSource: string | null;
utmCampaign: string | null;
utmMedium: string | null;
utmContent: string | null;
utmTerm: string | null;
urlParams: Record<string, string> | null;

nomeProduto?: string | null;
descricaoProduto?: string | null;
};

```

Observação de integração: aceite moeda como `number | string`. No primeiro formato, `valorPago` é o valor da venda inicial; no segundo, é o valor da invoice de renovação. A string normalmente contém um código numérico, como "1"; registros anteriores também podem usar "BRL".

6. Entrega e segurança

Esta seção reúne os detalhes necessários para validar chamadas, tratar falhas ou desenvolver um receptor próprio.

Formato da requisição

A VendePay envia uma requisição HTTP POST, com body JSON, para a URL cadastrada:

```

POST <URL configurada>
Content-Type: application/json
x-signature-key: <secret do webhook>
X-Webhook-Event: <identificador do evento>
X-Webhook-Id: <UUID do webhook>

```

Header	Descrição
Content-Type	Sempre application/json.
x-signature-key	Secret compartilhado, gerado quando o webhook é cadastrado.
X-Webhook-Event	Identificador do evento entregue.
X-Webhook-Id	Identificador da integração de webhook cadastrada na VendePay.

X-Webhook-Id identifica a configuração, não uma entrega individual. O mesmo valor aparece nas notificações enviadas pela mesma integração.

Valide o secret

Se a plataforma de destino permite consultar os headers recebidos, compare x-signature-key com o secret salvo durante o cadastro antes de executar ações na automação.

Mantenha a URL de recebimento e o secret privados. Se a plataforma não permite validar headers, revise as opções de autenticação, filtro ou gateway disponíveis antes de processar ações sensíveis.

Resposta esperada

Uma resposta HTTP na faixa 2xx é interpretada como sucesso. Não existe um formato obrigatório para o body da resposta.

Plataformas de automação normalmente respondem à chamada automaticamente. Em uma implementação própria, aceite ou registre a notificação rapidamente, retorne 2xx e deixe tarefas demoradas para depois da resposta.

Comportamento da entrega

- Entregas que falham não são reenviadas automaticamente.
- A ordem entre eventos não é garantida.
- Reenvios manuais podem ocorrer e não precisam ser idênticos à entrega original.
- Use o tipo do evento junto aos identificadores de compra, checkout ou invoice para evitar ações duplicadas.
- Não use somente X-Webhook-Id para identificar uma entrega, pois ele representa a configuração do webhook.

7. Solução de problemas

- **O teste não chegou:** confirme se a URL foi copiada por completo, está pública e continua ativa. Verifique também se o evento foi selecionado no webhook.
- **A plataforma não mostra os campos:** confirme se o body foi interpretado como JSON e faça um novo envio de teste enquanto a plataforma estiver aguardando uma amostra.
- **O teste chegou, mas o evento real não:** confirme se o evento e o produto estão incluídos na configuração. Sem produtos selecionados, o webhook recebe eventos de todos os produtos.
- **O payload real difere do teste:** os valores do teste são sintéticos. Considere os campos opcionais e anuláveis descritos na referência do evento.
- **A automação executou mais de uma vez:** crie uma regra de duplicidade usando o campo event e o identificador da compra, checkout ou invoice.
- **A chamada foi rejeitada:** verifique o status HTTP retornado, as regras de acesso do destino e o tempo necessário para responder. Falhas não são reenviadas automaticamente.
- **O secret não foi salvo:** cadastre uma nova integração para obter outro secret e, depois de validar a substituição, remova a configuração anterior.
- **A URL de recebimento mudou:** atualize a URL cadastrada na VendePay e envie um novo teste antes de retomar a automação.

8. Valores de referência

Método de pagamento

Valor	Método
1	PIX
2	Boleto
3	Cartão de crédito
4	PayPal
5	Apple Pay
6	Google Pay
7	Amazon Pay

Moeda da venda

Valor	Moeda	Valor	Moeda	Valor	Moeda
1	BRL	6	COP	11	DKK
2	USD	7	MXN	12	NOK
3	EUR	8	CAD	13	SEK
4	CLP	9	JPY	14	PYG
5	PEN	10	GBP	15	CHF

Alguns eventos de assinatura enviam a moeda como texto. Nos registros atuais, esse texto normalmente contém o código numérico da tabela, como "1" para BRL; registros anteriores também podem usar códigos como "BRL". Essas exceções estão indicadas em cada evento.

Status da venda

Valor	Status
1	Pendente
2	Paga
3	Recusada
4	Cancelada
5	Chargeback
6	Reembolso
7	Falha
8	Teste
9	Reembolso concluído

Status do checkout

Valor	Status
1	Pendente
2	Pago
3	Expirado
4	Falha